



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 5
по курсу «Языки и методы программирования»
«Stream API в Java»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Теоретические сведения	1
Задание 1: Класс StringSet	1
Класс StringSet	2
Метод countOccurrences()	2
Тестирование StringSet	3
Задание 2: Класс Ints (дополнительное)	3
Класс Ints	4
Метод unitPositions()	4
Метод findX()	4
Тестирование Ints	5
Вывод	5

Цель работы

Изучить Stream API в Java для функциональной обработки данных. Реализовать классы с использованием потоков для работы со строками и битовыми операциями.

Задачи:

- Изучить Stream API и методы stream().
- Реализовать класс StringSet для фильтрации строк.
- Реализовать класс Ints для работы с битовыми masks.
- Использовать IntStream и OptionalInt.

Теоретические сведения

Stream API — это мощный инструмент для функциональной обработки коллекций в Java. Основные методы:

- filter() — фильтрация элементов
- map() — преобразование элементов
- flatMap() — преобразование элементов
- forEach() — итерация по элементам
- reduce() — агрегация элементов
- findFirst(), findAny() — поиск элемента

Задание 1: Класс StringSet

Класс для работы со множеством строк с использованием Stream API.

Класс StringSet

StringSet.java

```
public class StringSet {
    private Set<String> strings;

    public StringSet(Set<String> strings) {
        this.strings = strings;
    }

    public Stream<String> stringStream(String s, int k) {
        return strings.stream()
            .filter(str → countOccurrences(str, s) == k);
    }

    public OptionalInt findN(String s) {
        if (strings.stream().filter(str → !
str.contains(s)).toList().size() == 0) {
            return strings.stream()
                .mapToInt(String::length)
                .min();
        }
        return strings.stream()
            .filter(str → !str.contains(s))
            .mapToInt(String::length)
            .max();
    }

    private int countOccurrences(String str, String sub) {
        if (sub.isEmpty() || str.isEmpty()) {
            return 0;
        }
        int count = 0;
        int idx = 0;
        while ((idx = str.indexOf(sub, idx)) != -1) {
            count++;
            idx += sub.length();
        }
        return count;
    }
}
```

Метод countOccurrences()

Метод подсчитывает количество вхождений подстроки в строку:

- Использует indexOf() для поиска каждого вхождения
- Сдвигает индекс на длину подстроки после каждого найденного вхождения

Тестирование StringSet

Test.java

```
public class Test {
    public static void main(String[] args) {
        new StringSet(new HashSet<>(Set.of("a", "aa", "aaa", "b",
"ab")))
            .stringStream("a", 1).forEach(System.out::println);
        System.out.println("");
        System.out.println(new StringSet(new HashSet<>(Set.of("a",
"aa", "aaa", "bbb", "ab"))).findN("a"));
        System.out.println(new StringSet(new HashSet<>(Set.of("a",
"aa", "aaa", "ab"))).findN("a"));
    }
}
```

Вывод:

- Строки, содержащие ровно 1 вхождение «a»: «b», «ab»
- findN(«a») для множества со «bbb» — максимальная длина = 3
- findN(«a») для множества без «bbb» — максимальная длина = 2

Задание 2: Класс Ints (дополнительное)

Класс для работы с битовыми операциями.

Класс Ints

Ints.java

```
public class Ints {
    private List<Integer> ints;

    public Ints(List<Integer> ints) {
        this.ints = ints;
    }

    public IntStream unitPositions() {
        return IntStream.range(0, ints.size())
            .flatMap(j → IntStream.range(0, 32)
                .filter(i → (ints.get(j) >> i & 1) == 1)
                .map(i → 32 * j + i));
    }

    public OptionalInt findX() {
        int mask = ints.stream().reduce(0, (a, b) → a | b);
        return IntStream.range(0, ints.size())
            .filter(i → (ints.get(i) | mask) == mask)
            .findFirst();
    }
}
```

Метод unitPositions()

Возвращает позиции установленных битов:

- Внешний цикл: по всем числам (0...size)
- Внутренний цикл: по битам (0...31)
- Фильтр: оставляет только установленные биты $(x \gg i \& 1) == 1$
- flatMap: объединяет все позиции в один поток

Формула: позиция = 32 умножить номер_числа плюс номер_бита

Метод findX()

Находит индекс первого числа, содержащего все биты:

- reduce(0, (a, b) → a | b) — вычисляет битовую маску всех чисел
- filter: оставляет числа, которые содержат все биты маски
- findFirst: возвращает первый индекс

Тестирование Ints

dop/Test.java

```
public class Test {
    public static void main(String[] args) {
        // [1, 3, 7, 16] в двоичном: [1, 11, 111, 10000]
        new Ints(Arrays.asList(1, 1 + 2, 1 + 2 + 4,
16)).unitPositions()
            .forEach(x → System.out.println(x));
        // [3, 5, 7, 1] → mask = 7 (111)
        Ints ints = new Ints(Arrays.asList(3, 5, 7, 1));
        OptionalInt result = ints.findX();
        if (result.isPresent()) {
            System.out.println(result.getAsInt());
        }
    }
}
```

Вывод unitPositions():

- 1 = 1 -> позиция 0
- 3 = 1 + 2 -> позиции 0, 1
- 7 = 1 + 2 + 4 -> позиции 0, 1, 2
- 16 = 10000 -> позиция 4 + 32 = 36

Вывод findX():

- mask = 3 | 5 | 7 | 1 = 7 (111 в двоичном)
- Число 3 (011) содержит все биты маски 7 (111)? Да
- Результат: индекс 0

Вывод

В ходе лабораторной работы были изучены и применены:

1. Stream API — функциональный подход к обработке данных в Java
2. Класс StringSet — фильтрация строк по количеству вхождений подстроки
3. Метод findN() — поиск минимальной или максимальной длины строки
4. Класс Ints — работа с битовыми масками
5. Методы unitPositions() и findX() — поиск позиций битов и индексов

Stream API позволяет писать лаконичный и выразительный код для обработки коллекций.